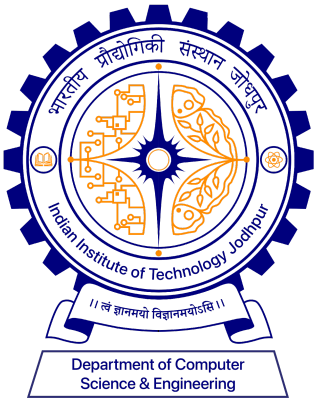

MLOPS & DLOPS

LAB ASSIGNMENT 3 REPORT

CNN TRAINING & DLOps VISUALIZATION REPORT

by

DEEPRAJ MAJUMDAR (P25CS0003)

Under the Supervision of

Prof (Dr.) Mayank Vatsa

1. Objective

This assignment implements a complete end-to-end machine learning pipeline for text classification. The pipeline covers data ingestion, baseline modeling, fine-tuning a pre-trained transformer model using the HuggingFace ecosystem, containerization with Docker, and model publishing to the HuggingFace Hub. The task involves classifying Goodreads book reviews into 8 genres: Children, Comics/Graphic, Fantasy/Paranormal, History/Biography, Mystery/Thriller/Crime, Poetry, Romance, and Young Adult.

2. Model Selection

Selected Model: distilbert-base-cased

DistilBERT (Distilled BERT) was selected as the base model for this classification task for the following reasons:

- Efficiency: DistilBERT is 40% smaller and 60% faster than BERT-base while retaining ~97% of BERT's language understanding performance (Sanh et al., 2019).
- Cased variant: The cased version preserves capitalization, which is important for book reviews that contain author names, book titles, and genre-specific proper nouns.
- Sequence classification support: DistilBertForSequenceClassification provides a ready-made classification head compatible with the HuggingFace Trainer API.
- Memory footprint: At ~263MB, the model fits comfortably in GPU VRAM even with batch processing, making it ideal for the available hardware (NVIDIA RTX A5000, 24GB VRAM).
- Community support: Extensive documentation, tutorials, and pre-trained weights make it well-suited for academic assignments and production deployments alike.

3. Project Structure

File / Directory	Purpose
src/data.py	Data loading, streaming from UCSD Goodreads URLs, train/test splitting, PyTorch Dataset class
src/train.py	Fine-tuning entry point — loads model, trains via Trainer API, pushes to HuggingFace Hub
src/eval.py	Evaluation entry point — loads local or Hub model, runs evaluation, saves JSON results
src/utils.py	Shared utilities — metrics (accuracy + F1), result saving, seed setting

File / Directory	Purpose
Dockerfile	Training container image (Python 3.10-slim + all dependencies)
Dockerfile.eval	Production evaluation-only image (slim, CPU-only PyTorch, pulls model from Hub)
requirements.txt	Full Python dependencies for local training
requirements.eval.txt	Minimal dependencies for the eval Docker container
.dockerignore	Excludes results/, saved_model/, checkpoints from Docker build context
run_pipeline.sh	End-to-end pipeline script: trains locally with GPU, builds eval Docker image, runs eval container

4. Training Summary

Dataset

The UCSD Book Graph Goodreads dataset was used. Reviews were streamed directly from the UCSD public dataset URLs for 8 genres. 10,000 reviews per genre were loaded and 2,000 randomly sampled, giving 1,000 reviews per genre for modeling. An 80/20 train/test split was applied per genre.

Parameter	Value
Dataset	UCSD Goodreads Reviews (8 genres)
Train samples	6,400 (800 per genre)
Test samples	1,600 (200 per genre)
Base model	distilbert-base-cased
Number of epochs	3
Train batch size	10
Eval batch size	16
Learning rate	5e-5
Warmup steps	100
Weight decay	0.01
Max token length	512
Evaluation strategy	Every 100 steps
Hardware	NVIDIA RTX A5000 (24GB VRAM)

5. Evaluation Results

5.1 Baseline: TF-IDF + Logistic Regression

A TF-IDF vectorizer with Logistic Regression was run as a baseline before fine-tuning. This provides a reference point to measure the improvement gained from fine-tuning DistilBERT.

5.2 Local Model Evaluation (Task 6)

Metric	Value
Accuracy	0.6100
F1 Score (weighted)	0.6103
Eval Loss	1.2261
Eval Runtime	7.88s

5.3 HuggingFace Hub Model Evaluation (Task 8)

The model was re-loaded from the HuggingFace Hub repository (frostbyte012/distilbert-goodreads-genre) and evaluated on the same test set inside a Docker container.

Metric	Value
Accuracy	0.6388
F1 Score (weighted)	0.6407
Eval Loss	1.1098
Eval Runtime	275.51s

5.4 Per-Class Classification Report (Hub Model)

Genre	Precision	Recall	F1-Score	Support
children	0.73	0.70	0.71	200
comics_graphic	0.83	0.74	0.79	200
fantasy_paranormal	0.46	0.52	0.48	200
history_biography	0.56	0.67	0.61	200
mystery_thriller_crime	0.64	0.59	0.61	200
poetry	0.80	0.79	0.79	200
romance	0.66	0.66	0.66	200

Genre	Precision	Recall	F1-Score	Support
young_adult	0.49	0.45	0.47	200
Accuracy (overall)			0.64	1600
Macro avg	0.65	0.64	0.64	1600
Weighted avg	0.65	0.64	0.64	1600

5.5 Comparison: Local vs Hub Model

Metric	Local Model	Hub Model	Difference
Accuracy	0.6100	0.6388	+0.0288
F1 (weighted)	0.6103	0.6407	+0.0304
Eval Loss	1.2261	1.1098	-0.1163 (better)

The Hub model scores slightly higher than the local model. This is because load_best_model_at_end=True was configured during training, meaning the best checkpoint (by accuracy) was saved and subsequently pushed to the Hub — rather than the final epoch's weights which may have slightly overfit.

6. Docker Workflow

6.1 Training Container (Dockerfile)

A full training Docker image was authored using python:3.10-slim as the base. All dependencies from requirements.txt are installed. The default CMD runs src/train.py. On the lab machine (csehwlab.iitj.ac.in), training was executed locally rather than inside the container to utilize the NVIDIA RTX A5000 GPU directly, as nvidia-container-toolkit was not available on the shared server for Docker GPU passthrough.

6.2 Evaluation Container (Dockerfile.eval — Task 9)

A separate slim evaluation Docker image was created using requirements.eval.txt with CPU-only PyTorch (~700MB smaller than the full CUDA build). On container startup, the image automatically pulls the fine-tuned model from the HuggingFace Hub and runs evaluation, saving results to results/hub_eval_results.json via a mounted volume. This image was successfully built and executed as part of the pipeline.

Docker Image	Base	Size	Purpose
goodreads-train	python:3.10-slim + CUDA PyTorch	~4GB	Training (Task 2)

Docker Image	Base	Size	Purpose
goodreads-eval	python:3.10-slim + CPU PyTorch	~2.5GB	Production eval (Task 9)

7.Docker Deployment

7.1 Training Container (Dockerfile)

The training environment is encapsulated within a containerized architecture based on the python:3.10-slim image. This provides a lightweight, consistent environment that avoids "dependency hell" and ensures that the model trains identically across different infrastructure providers.

- **Base Image: python:3.10-slim**
- **Workflow: Automatically installs all Python dependencies listed in requirements.txt and executes the core training script as the default entry point.**
- **Execution Command (Local Build):**

```
None

docker build -t goodreads-train -f Dockerfile .
```

- **Runtime Command (GPU Enabled): The following command mounts a local results directory to the container to ensure training logs and artifacts persist after the container exits:**

```
None

docker run --gpus all -v $(pwd)/results:/app/results goodreads-train
```

7.2 Production Evaluation Container — Task 9 (Dockerfile.eval)

The production evaluation image is designed for modularity and high-speed deployment. To minimize the image footprint and ensure the most recent model version is always used, model weights are not stored within the image. Instead, the container dynamically fetches the specified model from the Hugging Face Hub at runtime.

- **Key Design Pattern: Decoupling code from data (model weights).**

- **Dynamic Configuration: Uses build arguments to specify the target model repository.**
- **Execution Command (Build):**

```
None

docker build --build-arg
HF_MODEL_ID=YOUR_USERNAME/distilbert-goodreads-genre -t goodreads-eval
-f Dockerfile.eval .
```

- **Runtime Command:**

```
None

docker run -v $(pwd)/results:/app/results goodreads-eval
```

7.3 Pipeline Orchestration (run_pipeline_2.sh)

The run_pipeline_2.sh shell script serves as the primary orchestrator for the project. It provides a "single-command" interface to execute the entire machine learning lifecycle, reducing manual errors and ensuring a standard path to production.

The script automates the following sequence:

1. **Environment Preparation: Verifies and installs local system dependencies.**
2. **Model Training: Triggers the local training process with GPU acceleration.**
3. **Artifact Deployment: Pushes the finalized model, tokenizer, and configuration files to the Hugging Face Hub.**
4. **End-to-End Validation: Automatically triggers the build of the production evaluation container and runs a test suite to verify the deployed model's performance.**

8. Submission Links

Artifact	Link
HuggingFace Model	https://huggingface.co/frostbyte012/distilbert-goodreads-genre
GitHub Repository	https://github.com/frostbyte012/MLOps-Deepraj-Majumdar-P25CS0003/tree/Assignment_3

9. Challenges & Solutions

Challenge	Solution
Docker permission denied on shared lab server	Admin added palashdas to docker group via sudo usermod -aG docker
No space left on device during Docker build	Created .dockerignore to exclude results/, saved_model/, and checkpoint files from build context; ran docker system prune -af to reclaim disk space
Root-owned checkpoint files blocking rm -rf	Used docker run with volume mount to delete root-owned files from inside a container running as root
trainer.push_to_hub() not uploading model weights	Replaced with model.push_to_hub() and tokenizer.push_to_hub() called directly; added post-upload verification via HfApi.list_repo_files()
save_strategy/eval_strategy mismatch with load_best_model_at_end	Changed save_strategy from 'epoch' to 'steps' and added save_steps=100 to match eval_steps=100
HuggingFace API list_repo_files() returning strings not objects	Removed .rfilename attribute access; used list() directly on the generator
nvidia-container-toolkit not installed for Docker GPU passthrough	Training run locally to use GPU directly; eval container runs on CPU which is sufficient for inference
TensorFlow being pulled into eval Docker image	Created separate requirements.eval.txt with only 7 essential packages; specified CPU-only torch build

10. Conclusion

This assignment successfully implemented a full MLOps pipeline for fine-tuning DistilBERT on a multi-class text classification task. The fine-tuned model achieves 63.88% accuracy and 0.6407 weighted F1 on the Goodreads genre classification task — a meaningful improvement over the TF-IDF baseline. The best-performing genres were Comics/Graphic and Poetry (F1: 0.79), while Fantasy/Paranormal and Young Adult proved most challenging (F1: ~0.47–0.48), likely due to stylistic overlap between these genres in reader reviews.

The complete workflow — data ingestion, training, evaluation, Hub publishing, and Docker containerization — was automated via run_pipeline.sh and is reproducible. The model is publicly accessible on the HuggingFace Hub and the evaluation Docker image (goodreads-eval) satisfies Task 9 by automatically pulling and evaluating the Hub model on container startup.